

Група 1

Зад. 1

Да се напише функция със сигнатура `char* findWord(const char* str, char c)`, която приема символен низ `str` и произволен символ `c`. Функцията трябва да връща динамично заделен низ с точен размер, съдържащ копие на думата от `str`, в която даденият символ `c` се среща най-много пъти. Ако съществуват няколко такива думи, функцията да върне първата срещната. Ако в низа не се среща даденият символ, функцията да върне нулев указател. „Дума“ наричаме последователност от символи, ограничени от произволен брой интервали (символ ' ') или началото или края на символния низ.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Зад. 2

Напишете функция със сигнатура `int split(const char* str, char* result[])`. Функцията трябва да разделя входния низ `str` на съставлящите го думи. Всяка дума трябва да бъде копирана в нов динамично заделен низ, а указател към него да се запише в масива от указатели `result`. Функцията да връща броя на намерените думи.

Пояснения: Считаме, че `result` е достатъчно голям, за да побере всички резултати.

Входният низ може да съдържа водещи, крайни или поредни интервали и или табулации, които трябва да бъдат игнорирани. „Дума“ наричаме последователност от символи, ограничени от произволен брой интервали (символ ' ') или началото или края на символния низ. След изпълнението на функцията, масивът `result` трябва да съдържа точно толкова указатели, колкото думи са намерени в низа, като всеки указател сочи към динамично заделена памет с копие на съответната дума.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Група 2

Зад. 1

Напишете функция `double* parseDecimals(const char* str, int& outputSize)`. Функцията приема символен низ `str`, състоящ се от произволен брой десетични дробни числа, разделени от точка и запетая (символ `' ; '`). Функцията трябва да превърне числата в числа с плаваща запетая в тип `double`, като върне динамично заделен масив с точен размер и запише размера му във входния параметър `outputSize`.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход (низ):

42.67438633240699;23.33048227968354

Възможен изход (масив от `double`, приблизително):

42.67438633240699

23.33048227968354

Зад. 2

Напишете функция със сигнатура `unsigned int encodeMatrix(int matrix[4][4])`, която приема като вход матрица с размер 4x4, състояща се от цели числа между 0 и 3. Функцията трябва да представи таблицата като единствено 32-битово число, в което всеки елемент от оригиналната матрица (прочетена отляво надясно, и отгоре надолу) е записан в 2 бита от числото.

Напишете програма с функция `main()`, която въвежда матрица като вход от потребителя, извиква функцията върху нея и отпечатва резултата.

Пример:

Вход:

1 0 3 1

2 3 2 1

0 2 0 1

2 3 0 3

Изход:

1303978419

Пояснение:

1303978419 = 0x4DB921B3 = **01001101101110010010000110110011**
1 0 3 1 2 3 2 1 0 2 0 1 2 3 0 3

Група 3

Зад. 1

Напишете функция със сигнатура `int* findLongestIncreasingSequence(int arr[], int size, int& outputSize)`, която приема като параметри масив от цели числа `arr` и неговия размер `size`. Функцията трябва да открие най-големия подмасив от последователни нарастващи елементи (всеки следващ е по-голям от предходния) и да го върне като нов динамично заделен масив с размер `outputSize`. Ако има няколко поредици с еднаква дължина, да се върне първата срещната.

Напишете програма с функция `main()`, която въвежда масив като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход:

9

1 2 2 3 4 1 2 3 0

Изход:

2 3 4

Зад. 2

Напишете функция със сигнатура `int extractSequences(const char* str)`, която приема символен низ, състоящ се от малки латински букви, цифри и символите `'.'` и `'*'`. Функцията трябва да преброи поднизовите на `str`, състоящи се от цифров символ, последван от толкова на брой еднакви букви. Освен букви, допустим е символът `'.'`, който може да се замени с една произволна буква, и символът `'*'`, който може да се замени с произволен брой букви. Приемаме, че в подадения низ нито един цифров символ няма да бъде последван от `'.'` или `'*'`.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход:

ab3cccc1ab9dd*d5e3f.f

Изход:

3 (валидни поднизове са: 3cccc, 9dd*d, 3f.f)

Група 4

Зад. 1

Напишете функция със сигнатура `char* encrypt(const char* str, int offset)`, която криптира (шифрова) входен символен низ `str`, съдържащ само малки латински букви и интервали, по следния начин. Интервалите остават непроменени. Всяка буква се криптира, като се замества с буквата, която се намира на `offset` на брой места надясно в латинската азбука. Например, за `offset=2`, буквата `'a'` се превръща в `'c'`, `'b'` – в `'d'`, ..., `'x'` става `'z'`, `'y'` става `'a'`, `'z'` става `'b'`. Ако обаче буквата, която ще бъде криптирана, има пореден номер в азбуката (`'a'` е 1, `'b'` е 2 и т.н., независимо от отместването), който се дели на отместването без остатък, буквата да не се премества, а вместо това да се дублира – например, при отместване 3, буквата `'c'` няма да се превърне в `'f'`, а в `"cc"`. Функцията да връща криптирания текст в нов динамично заделен символен низ с точен размер.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход:

```
str = "this is a secret message"
offset = 5
```

Изход:

```
ttmnx nx f xeehweett reexxflee
```

Зад. 2

Напишете функция със сигнатура `char* extractDuplicates(const char* str)`, която приема символен низ `str`. Функцията трябва да върне нов динамично заделен низ, съдържащ само символите, които се срещат повече от веднъж в оригиналния низ. Редът на символите във върнатия низ трябва да бъде лексикографски сортиран (възходящо по ASCII). Върнатият низ трябва да е с точно заделена памет за съдържанието му. Ако няма дублирани символи, върнете празен низ.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход	Възможен изход
programming	gmr

hello world	lo
abcdef	
aabbcc	abc

Група 5

Зад. 1

Напишете функция със сигнатура `int* collectPerfectNumbers(int** matrix, int rows, int cols, int& resultSize)`. Функцията да връща като резултат динамично заделен (едномерен) масив с точен размер `resultSize`, съдържащ онези елементи на целочислената матрица `matrix` с размерност `rowsxcols`, които са *перфектни числа*. Перфектно число наричаме число, което е равно на сумата на своите делители (с изключение на самото число).

Напишете програма с функция `main()`, която въвежда масив като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход:

```
2 4
12 40 28 6
496 123 340 12
6 30 30 40
```

Изход:

```
28 6 496 6
```

Упътване:

```
6 = 1 + 2 + 3
28 = 1 + 2 + 4 + 7 + 14
496 = 1 + 2 + 3 + ... + 29 + 30 + 31
```

Зад. 2

Напишете функция със сигнатура `char* invertEvenWords(const char* str)`, която приема символен низ `str` и връща нов динамично заделен символен низ, съдържащ единствено думите от `str`, които имат четна дължина, обърнати наобратно. Входният низ може да съдържа водещи, крайни или поредни интервали и или табулации, които трябва да бъдат игнорирани.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход:

```
I went to the FMI building yesterday. He likes coding and reading at
FMI 5th floor.
```

Изход:

```
tnew ot gnidliub gnidoc ta .roolf
```

Група 6

Зад. 1

Да се напише функция със сигнатура `char* toBinaryString(const char* str)`, която приема символен низ `str`. Функцията трябва да състави нов символен низ, съдържащ за всеки символ от `str` неговото двоично представяне (съгласно ASCII кодовете на съставлящите го символи), като между отделните символи се поставя разделителят '^'. Върнатият низ трябва да бъде динамично заделен с точен размер.

Напишете програма с функция `main()`, която въвежда символен низ като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Вход:

AB

Изход:

01000001^01000010

Пояснение:

ASCII кодовете на символите 'A' и 'B' са съответно:

- 'A' → 65 → 01000001
- 'B' → 66 → 01000010

Зад. 2

Напишете функция със сигнатура `int* sumColumns(int** jagged, int rows, int columns[])`, която приема „назъбен“ двумерен масив `jagged`, в който всеки ред има различен брой колони. Масивът има `rows` на брой реда, а `i`-тият ред се състои от `columns[i]` на брой елемента. Функцията трябва да пресметне сбора на всички колони в `jagged`, като върне нов динамично заделен масив, съдържащ сумите. Приемаме, че в колона, която не съществува в даден ред, има стойност 0.

Напишете програма с функция `main()`, която въвежда масив като вход от потребителя, извиква функцията върху него и отпечатва резултата.

Пример:

Масив	Представяне	Изход
5 3 5 3 9 1 3 2 9 1 4 5	rows = 6 columns[0] = 3 columns[1] = 5 columns[2] = 4	21 15 19 8 2

1 2 9 3	<pre>columns[3] = 0 columns[4] = 3 columns[5] = 1 jagged[0] = {5, 3, 5} jagged[1] = {3, 9, 1, 3, 2} jagged[2] = {9, 1, 4, 5} jagged[3] = {} (или nullptr) jagged[4] = {1, 2, 9} jagged[5] = {3}</pre>	
------------	--	--